```
Array
(
    [0] => argv_argc.php
    [1] => some
    [2] => data
)
Array
(
    [0] => argv_argc.php
    ...
```

A command-line shell will pass any text written after the command itself via a standard mechanism (`STDIN`); each item is delimited by some kind of space or tab character. Any program running in this way can access this data as *arguments* (a mechanism for passing data to functions, which we'll see in Chapter 3, "Functions," and Chapter 6, "Object Fundamentals"). In our case, the command is `php`, and the arguments are the filename `argv_argc.php`, `some`, and `data`. As you can hopefully see from the output of the program, we have two ways to access the argument values: either directly via `$argv` or as the `argv` element of the `$_SERVER` array. A count of the available arguments is provided via `$argc`. `$argc` and `$argv` are automatically created and made available everywhere in the program, and `$_SERVER` is a special type of array variable called a *superglobal*, which will have an `argc` and `argv` element populated with the same data. Superglobals are covered in Chapter 2, "Data, Types, and Variables".

In addition to arguments, CLI binaries can have options set using a mechanism known as *switches*. These take the form of `-x` (a "short" option) or `--name_of_x` (a "long" option) and can also be accessed in PHP using the core `getopt()` function. This function has a range of behaviors that might not be completely intuitive at first glance, so let's go step-by-step. First of all, here's a very simple form of the function that checks to see if a short `-o` has been supplied:

```
// getopt1.php
var_dump(getopt("o"));
```

Here's what happens when we call that code with and without the option:

```
$ php getopt1.php
array(0) {
}
```